LangSmith Observability: Trace, Monitor, and Improve LLM Apps

A comprehensive guide to instrumenting your LLM applications – from first traces to production-grade monitoring, evaluation, and debugging – all in one platform.

OBSERVABILITY MONITORING DEBUGGING

Author - Sambita Chakraborty

What LangSmith Does (in One Line)

LangSmith is the single platform for end-to-end LLM application observability – from prototype to production. It connects all the dots between what your code sends, what the model returns, and how your users experience the result.

Instrument

Wrap your LLM pipeline to capture every step automatically – no manual logging required.

Investigate

Dive into traces to understand exactly what happened at each stage of a request.

Monitor

Track production performance end-to-end with dashboards, alerts, and automated evaluations.

> Instrument your LLM application to investigate traces and monitor production performance end-to-end.

Core Building Blocks You'll See in LangSmith

Understanding LangSmith's vocabulary is the key to navigating its UI and API effectively. Every concept maps to a real artifact in your application's execution model.

1

Project

Groups all traces belonging to a single application or service. Think of it as a workspace scoped to one deployable unit.

2

Trace

Represents one complete operation – such as a single user request flowing through your entire pipeline from start to finish.

3

Run

A unit of work within a trace: an LLM call, a retrieval step, a parsing operation, or any discrete function execution.

4

Thread

Links multi-turn conversation traces using a shared identifier – `session_id`, `thread_id`, or `conversation_id`.

Get Started: Account, API Key, and Tracing Basics

Getting up and running with LangSmith takes just a few minutes. No credit card is required to create an account – head to **smith.langchain.com** and sign up instantly. Once inside, generating your first API key is straightforward.

Navigate to **Settings → API Keys → Create API Key**, copy it immediately, and store it in a secure secrets manager or environment file. With your key in hand, you have three flexible paths to enable tracing in your app.

→ Environment Variables

Set `LANGSMITH_API_KEY` and `LANGSMITH_TRACING` before running your app.

→ SDK Setup

Use the LangSmith Python, TypeScript, Kotlin, or Java SDK to configure tracing programmatically.

→ Integrations

Plug into LangChain, OpenAI wrappers, or other supported frameworks with zero extra code.

Quick Setup Checklist

- ○ Create account at smith.langchain.com
- ○ Navigate to Settings → API Keys
- ○ Click Create API Key
- ○ Copy and store key securely
- ○ Set environment variables
- ○ Run your first traced request

Add Tracing Quickly (Quickstart Promise)

LangSmith is designed for zero-friction onboarding. The moment you wrap your pipeline, every LLM call and tool invocation appears as a unified, nested trace – no restructuring of your existing code needed.

Complete Trace Records

LangSmith captures every step – from raw inputs to final outputs – as a structured record with timing, token counts, and metadata.

Nested Spans

Wrap your pipeline so tool calls and LLM calls appear together as one parent trace with clearly nested child spans for each operation.

Multi-Language Wrappers

Use the **OpenAI wrapper** or **traceable decorator** in Python, TypeScript, Kotlin, or Java to instrument any stack.

Trace an LLM App Pipeline (What "Good Tracing" Looks Like)

Good tracing isn't just about catching errors – it's about building institutional knowledge of how your app behaves at every lifecycle stage. LangSmith gives you consistent visibility whether you're in a Jupyter notebook or serving millions of requests.

1. **Prototyping**

   Trace early experiments to understand baseline behavior and iterate on prompts rapidly.

2. **Beta Testing**

   Capture real user interactions to identify edge cases before full rollout.

3. **Production**

   Monitor live traffic continuously – inspect prompts, responses, and latency without changing code.

> Trace LLM calls first to get immediate value, then progressively expand tracing to cover retrieval, parsing, and tool-use steps across your full pipeline.

Use LangChain Integration (Zero-Extra-Code Logging)

Two Variables, Full Tracing

Set these environment variables before running any LangChain application:

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=your_key_here
LANGSMITH_PROJECT=my-app
```

That's it. Every chain, agent, and tool invocation is automatically captured.

How It Works

LangSmith integrates seamlessly with LangChain – run your existing code normally and traces are logged automatically behind the scenes. No decorators, no wrappers, no code changes whatsoever.

- All traces go to a **default** project unless you specify otherwise
- Set `LANGSMITH_PROJECT` to route traces to a custom named project
- Works with LangChain Expression Language (LCEL), agents, and legacy chains
- Compatible with async and streaming LangChain workloads

Running Your Traced .py File Locally

01

Install LangSmith SDK

`pip install langsmith`

02

Create a .env file

Add `LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY=your_key_here`, and `LANGSMITH_PROJECT=my-app`.

03

Load env variables

Use python-dotenv: `from dotenv import load_dotenv; load_dotenv()`

04

Run your script

`python your_script.py` in the terminal.

05

View traces

Open `smith.langchain.com` and navigate to your project to see live traces.

Alternative: Run with LangGraph Dev Server

01

Create langgraph.json

Create a `langgraph.json` file in your project root with the following content:

```
{
  "dependencies": ["."],
  "graphs": {
    "agent":
"./your_agent.py:graph"
  },
  "env": ".env"
}
```

Replace `your_agent.py:graph` with the path to your graph object.

02

Install LangGraph CLI

Run `pip install langgraph-cli` in your terminal.

03

Start the dev server

Run `langgraph dev` in your project root. This spins up a local LangGraph server with hot-reloading enabled.

04

Open LangGraph Studio

The CLI will print a Studio URL (e.g. `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`). Open it in your browser.

05

Traces auto-appear

All runs triggered via the dev server are automatically traced to LangSmith under the project defined in your `.env` file.

`langgraph dev` is ideal for iterating on agent logic locally – it gives you a visual Studio UI, live reloading, and automatic LangSmith tracing without any extra setup.

Trace Selectively and Organize Results

Not every operation needs to be traced with equal granularity. LangSmith gives you fine-grained control over what gets captured, how it's tagged, and how multi-turn conversations are stitched together for analysis.

Selective Tracing

Use LangSmith's tracing mechanisms to capture only the steps that matter – reducing noise and storage costs without losing visibility on critical paths.

Metadata & Tags

Attach custom metadata to traces – user IDs, model versions, feature flags – so you can filter and slice your data into meaningful segments during analysis.

Thread Grouping

Group multi-turn interactions into a single thread by passing the correct metadata key. Use `session_id`, `thread_id`, or `conversation_id` to link related traces automatically.

Improve Quality in Studio: Iterate, Evaluate, and Debug

LangSmith Studio goes far beyond passive logging – it's an active workspace for improving your application's behavior. Use it to close the feedback loop between observed failures and prompt or logic changes.

Inspect Traces

Browse prompts, responses, datasets, and experiments in an intuitive UI without writing any queries.

Run Experiments

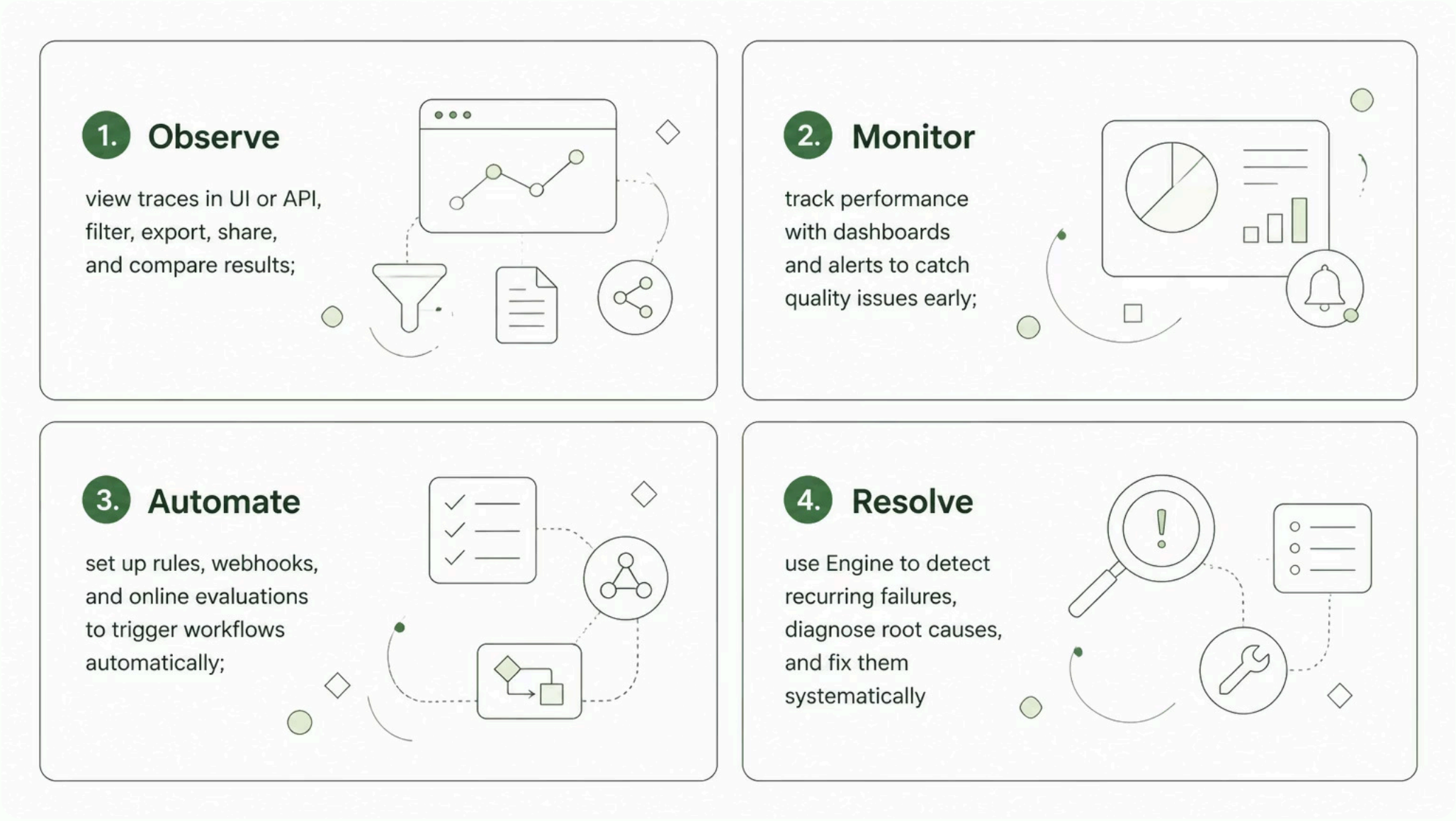
Modify prompts and run them over a curated dataset to score outputs and compare results side-by-side.

Import & Debug

Pull traced runs directly into Studio to reproduce edge cases, diagnose failures, and refine behavior iteratively.

Production Playbook: Monitor, Automate, and Fix Failures

LangSmith transforms production operations from reactive firefighting into proactive quality engineering. Combine dashboards, automation rules, and AI-powered root cause analysis to keep your LLM app healthy at scale.



With LangSmith's full production playbook in place, your team can catch regressions before users notice, automate quality gates, and continuously improve model behavior – all from a single platform.

Reference:**https://docs.langchain.com/langsmith/observability**